

Ticketing System - Complete Project Summary

Version: 1.0.0

Date: 2024

Project Type: Full-Stack Event Ticketing System

Table of Contents

- [Project Description](#)
- [Technology Stack](#)
- [System Architecture](#)
- [Package Dependencies](#)
- [Features](#)
- [System Flow](#)
- [Infrastructure](#)
- [Provisioning](#)
- [Database Schema](#)
- [Security](#)
- [Scalability](#)

1. Project Description

The **Ticketing System** is a comprehensive event management and ticket purchasing platform designed to handle event listings, user authentication, ticket sales, and administrative functions. The system provides a scalable, secure, and production-ready solution for managing events and ticket purchases.

Key Characteristics

- Full-Stack Application:** React frontend with Node.js/Express backend
- RESTful API:** REST API architecture with JWT-based authentication
- Database:** PostgreSQL for data persistence
- Containerized:** Docker-based deployment with Docker Compose
- Production-Ready:** Load balancing, health checks, monitoring capabilities
- Scalable:** Stateless design supporting horizontal scaling
- Secure:** Role-based access control, password hashing, rate limiting

Use Cases

- Event organizers can create and manage events
- Users can browse events, register accounts, and purchase tickets
- Administrators can manage users, events, and system settings
- System supports concurrent ticket purchases with atomic transactions

2. Technology Stack

Frontend

Technology	Version	Purpose
React	18.2.0	UI framework for building interactive user interfaces
TypeScript	5.2.2	Type-safe JavaScript for enhanced development experience
Vite	5.0.8	Fast build tool and development server
React Router DOM	6.20.0	Client-side routing for single-page application

Axios	1.6.2	HTTP client for API communication
-------	-------	-----------------------------------

Backend

Technology	Version	Purpose
Node.js	Latest LTS	JavaScript runtime environment
Express	5.1.0	Web application framework for RESTful APIs
TypeScript	5.5.0	Type-safe JavaScript for backend development
PostgreSQL	15-alpine	Relational database management system
pg	8.11.3	PostgreSQL client for Node.js

Infrastructure & DevOps

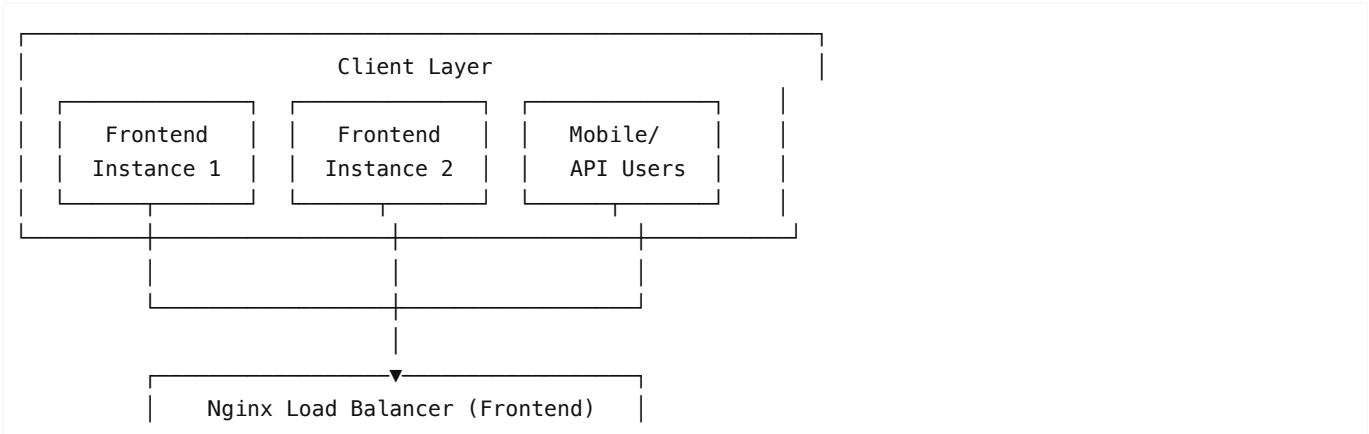
Technology	Purpose
Docker	Containerization platform
Docker Compose	Multi-container orchestration
Nginx	Load balancer and reverse proxy
Bash Scripts	Infrastructure provisioning and automation

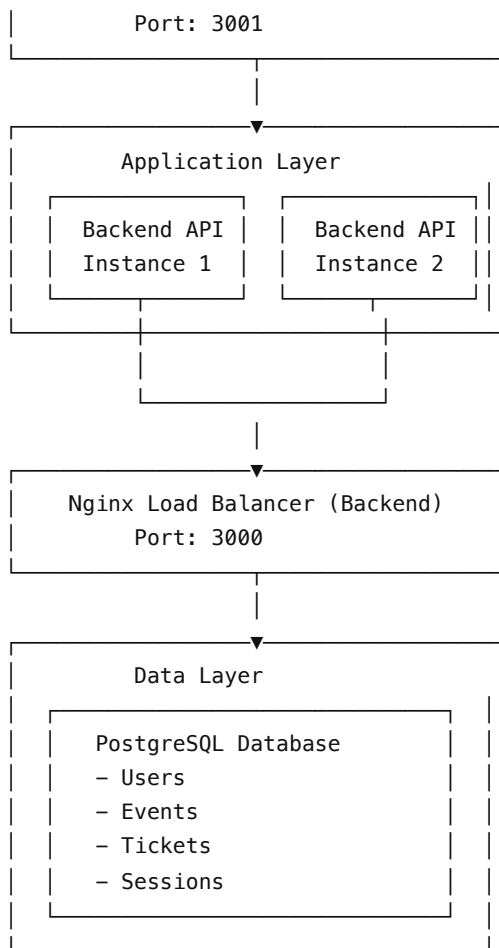
Security & Authentication

Technology	Version	Purpose
jsonwebtoken	9.0.2	JWT token generation and verification
bcryptjs	2.4.3	Password hashing with bcrypt algorithm
helmet	7.1.0	Security headers middleware
express-rate-limit	7.1.5	Rate limiting to prevent abuse
express-validator	7.0.1	Input validation and sanitization

3. System Architecture

High-Level Architecture





Application Architecture (Clean Architecture Pattern)

```
backend/
├── src/
│   ├── index.ts           # Application entry point
│   ├── db/                # Data Access Layer
│   │   ├── connection.ts  # Database connection pool
│   │   ├── migrations.ts  # Schema initialization
│   │   └── seed.ts        # Database seeding
│   ├── models/            # Domain Models & DTOs
│   │   ├── User.ts
│   │   ├── Event.ts
│   │   ├── Ticket.ts
│   │   ├── Auth.ts
│   │   └── Session.ts
│   ├── services/          # Business Logic Layer
│   │   ├── authService.ts
│   │   ├── userService.ts
│   │   ├── eventService.ts
│   │   ├── ticketService.ts
│   │   └── sessionService.ts
│   ├── controllers/       # Request Handling Layer
│   │   ├── authController.ts
│   │   ├── userController.ts
│   │   ├── eventController.ts
│   │   └── ticketController.ts
│   └── routes/            # Routing Layer
```

```
| | | | authRoutes.ts
| | | | userRoutes.ts
| | | | eventRoutes.ts
| | | | ticketRoutes.ts
| | | |
| | | | middleware/          # Cross-cutting Concerns
| | | | | auth.ts           # JWT authentication & authorization
| | | | |
| | | | | utils/           # Utility Functions
| | | | | validation.ts     # Input validation helpers
```

Frontend Architecture

```
frontend/
├── src/
|   ├── main.tsx          # Application entry point
|   ├── App.tsx           # Root component with routing
|   ├── pages/            # Page Components
|   |   ├── LoginPage.tsx
|   |   ├── RegistrationPage.tsx
|   |   ├── EventsPage.tsx
|   |   ├── PurchasePage.tsx
|   |   ├── AdminEventsPage.tsx
|   |   └── AdminUsersPage.tsx
|   ├── components/       # Reusable Components
|   |   └── ProtectedRoute.tsx
|   ├── context/          # React Context
|   |   └── AuthContext.tsx # Authentication state management
|   ├── services/         # API Client Services
|   |   ├── apiClient.ts  # Axios instance with interceptors
|   |   ├── authService.ts
|   |   ├── eventService.ts
|   |   └── userService.ts
|   ├── utils/            # Utility Functions
|   |   ├── tokenStorage.ts # Token management
|   |   ├── apiConfig.ts   # API configuration
|   |   └── validation.ts  # Form validation
|   └── types/            # TypeScript Type Definitions
|       └── index.ts
```

4. Package Dependencies

Backend Dependencies

Production Dependencies

Package	Version	Purpose
express	^5.1.0	Web framework for building REST APIs
cors	^2.8.5	Cross-Origin Resource Sharing middleware
dotenv	^16.4.5	Environment variable management
pg	^8.11.3	PostgreSQL client library with connection pooling
jsonwebtoken	^9.0.2	JWT token creation and verification
bcryptjs	^2.4.3	Password hashing with bcrypt (10 salt rounds)

express-validator	^7.0.1	Request validation and sanitization
helmet	^7.1.0	Security headers (XSS, CSRF, clickjacking protection)
express-rate-limit	^7.1.5	Rate limiting to prevent API abuse

Development Dependencies

Package	Version	Purpose
typescript	^5.5.0	TypeScript compiler
ts-node-dev	^2.0.0	TypeScript development server with hot reload
@types/express	^5.0.3	TypeScript definitions for Express
@types/cors	^2.8.17	TypeScript definitions for CORS
@types/node	^20.14.0	TypeScript definitions for Node.js
@types/pg	^8.10.9	TypeScript definitions for PostgreSQL client
@types/jsonwebtoken	^9.0.5	TypeScript definitions for JWT
@types/bcryptjs	^2.4.6	TypeScript definitions for bcrypt

Frontend Dependencies

Production Dependencies

Package	Version	Purpose
react	^18.2.0	UI library for building user interfaces
react-dom	^18.2.0	React DOM renderer
react-router-dom	^6.20.0	Declarative routing for React applications
axios	^1.6.2	Promise-based HTTP client for API requests

Development Dependencies

Package	Version	Purpose
typescript	^5.2.2	TypeScript compiler
vite	^5.0.8	Fast build tool and dev server
@vitejs/plugin-react	^4.2.1	Vite plugin for React support
@types/react	^18.2.43	TypeScript definitions for React
@types/react-dom	^18.2.17	TypeScript definitions for React DOM
eslint	^8.55.0	JavaScript/TypeScript linter
@typescript-eslint/parser	^6.14.0	TypeScript parser for ESLint
@typescript-eslint/eslint-plugin	^6.14.0	ESLint plugin for TypeScript
eslint-plugin-react-hooks	^4.6.0	ESLint rules for React Hooks
eslint-plugin-react-refresh	^0.4.5	ESLint plugin for React Refresh

5. Features

5.1 Authentication & Authorization

User Registration

- Email-based registration
- Password strength validation
- Automatic user role assignment (default: 'user')
- Email uniqueness enforcement
- Password hashing with bcrypt (10 rounds)

User Login

- Email/password authentication
- JWT token generation
- Token expiration management (configurable, default: 7 days)
- Session tracking with device info and IP address
- Automatic token refresh support

User Logout

- Token invalidation
- Session revocation
- Expired session cleanup (hourly job)

Role-Based Access Control (RBAC)

- **User Role:** Can manage own profile, browse events, purchase tickets
- **Admin Role:** Full system access, user management, event management

5.2 Event Management

Public Event Features

- **List Events:** Browse all available events
- **Filter Events:** Filter by status (upcoming, live, completed, cancelled)
- **Search Events:** Filter by venue, date range
- **View Event Details:** Full event information including ticket availability
- **Pagination:** Efficient data loading for large event lists

Authenticated Event Features

- **Create Events:** Authenticated users can create events
- **Update Events:** Event creators can update their events
- **Event Status Management:** Automatic status updates based on event date

Admin Event Features

- **Delete Events:** Admin-only event deletion
- **Full Event Management:** Complete CRUD operations
- **Event Analytics:** View event statistics

5.3 Ticket Purchase System

Ticket Purchase Flow

1. User selects event and quantity
2. System validates ticket availability
3. Atomic transaction ensures no over-booking
4. Ticket records created with purchase details
5. Available ticket count decremented
6. Purchase confirmation returned

Features

- **Atomic Transactions:** Prevents race conditions and over-booking
- **Real-time Availability:** Current ticket count displayed
- **Price Calculation:** Automatic price calculation based on quantity
- **Purchase History:** Users can view their ticket purchases
- **Transaction Safety:** Database-level constraints ensure data integrity

5.4 User Management

Self-Service Features

- View own profile
- Update own profile information
- Change password (with validation)
- View purchase history

Admin User Management

- List all users
- View user details
- Create new users
- Update user information
- Delete users
- Manage user roles
- Activate/deactivate users

5.5 Support Ticket System

Ticket Creation

- Create support tickets
- Set ticket priority (low, medium, high, urgent)
- Assign tickets to users
- Track ticket status (open, in_progress, resolved, closed)

Ticket Management

- List tickets with filtering
- Update ticket status
- Assign tickets to team members
- Delete tickets (admin only)

5.6 API Features

RESTful Design

- Standard HTTP methods (GET, POST, PUT, DELETE)
- RESTful URL structure
- JSON request/response format
- Consistent error handling

Security Features

- JWT-based authentication
- Role-based authorization
- Rate limiting (100 req/15min general, 5 req/15min auth)
- CORS protection
- Security headers (Helmet)
- Input validation and sanitization

Performance Features

- Database connection pooling
- Query optimization with indexes

- Pagination support
 - Efficient data loading
-

6. System Flow

6.1 User Registration Flow

1. User submits registration form
 - ↳ Frontend validates input
 - ↳ POST /api/auth/register
 - ↳ Backend validates email uniqueness
 - ↳ Hash password with bcrypt
 - ↳ Create user record in database
 - ↳ Return success response
 - ↳ Redirect to login page

6.2 User Login Flow

1. User submits login credentials
 - ↳ Frontend validates input
 - ↳ POST /api/auth/login
 - ↳ Backend validates credentials
 - ↳ Verify email exists
 - ↳ Compare password hash
 - ↳ Generate JWT token
 - ↳ Create session record
 - ↳ Return token + user info
 - ↳ Frontend stores token
 - ↳ Redirect to events page

6.3 Ticket Purchase Flow

1. User browses events (GET /api/events)
 - ↳ Frontend displays events
 - ↳ User selects event
 - ↳ Navigate to purchase page
 - ↳ GET /api/events/:id
 - ↳ Display event details
 - ↳ User selects quantity
 - ↳ POST /api/events/:eventId/purchase
 - ↳ Validate authentication
 - ↳ Check ticket availability
 - ↳ Begin database transaction
 - ↳ Create ticket records
 - ↳ Decrement available tickets
 - ↳ Commit transaction
 - ↳ Return purchase confirmation
 - ↳ Display success message

6.4 Event Creation Flow

1. Authenticated user navigates to create event
 - ↳ Frontend displays event form
 - ↳ User fills event details


```
└─> POST /api/events
    ├──> Validate authentication
    ├──> Validate request data
    ├──> Insert event record
    └─> Return created event
        └─> Display success message
```

6.5 Request Flow (Authenticated API Call)

```
1. Frontend makes API request
└─> Axios interceptor adds Authorization header
    └─> Request reaches Nginx Load Balancer
        └─> Load balancer routes to backend instance
            └─> Express middleware stack
                ├──> CORS middleware
                ├──> Helmet (security headers)
                ├──> Rate limiter
                ├──> Body parser
                └─> Route handler
                    ├──> authenticate middleware
                    │   ├──> Extract JWT token
                    │   ├──> Verify token signature
                    │   ├──> Check token expiration
                    │   └─> Attach user to request
                    ├──> authorize middleware (if required)
                    │   └─> Check user role
                    └─> Controller handler
                        ├──> Validate request data
                        ├──> Call service layer
                        │   ├──> Business logic
                        │   └─> Database operations
                        └─> Return response
                            └─> Frontend receives data
```

7. Infrastructure

7.1 Container Architecture

The system uses Docker Compose to orchestrate multiple services:

Services

- 1. PostgreSQL Database** (ticketing-postgres)
 - Image: postgres:15-alpine
 - Port: 5432 (internal)
 - Volume: postgres_data (persistent storage)
 - Health checks: pg_isready every 10 seconds
 - Auto-restart: unless-stopped
- 2. Backend API Instances** (ticketing-backend-1, ticketing-backend-2)
 - Built from: ./backend/Dockerfile
 - Port: 3000 (internal, no external exposure)
 - Environment: Production configuration
 - Resource limits: 1 CPU, 512MB RAM
 - Health checks: HTTP /health endpoint

- Auto-restart: unless-stopped

3. Nginx Load Balancer (Backend) (ticketing-nginx-lb-backend)

- Image: nginx:alpine
- Port: 3000 (external), 8080 (health check)
- Configuration: Least connection load balancing
- Features: Rate limiting, gzip compression, security headers

4. Frontend Instances (ticketing-frontend-1, ticketing-frontend-2)

- Built from: ./frontend/Dockerfile
- Served via: Nginx (static files)
- No external ports (accessed via load balancer)
- Resource limits: 0.5 CPU, 256MB RAM

5. Nginx Load Balancer (Frontend) (ticketing-nginx-lb-frontend)

- Image: nginx:alpine
- Port: 3001 (external), 8081 (health check)
- Configuration: Round-robin load balancing

7.2 Network Architecture

```
ticketing-network (bridge network)
├─ All services isolated in private network
├─ Only load balancers expose ports externally
└─ Internal service communication via service names
```

7.3 Storage Architecture

- **PostgreSQL Data:** Persistent Docker volume (postgres_data)
- **Backup Strategy:** Manual backup scripts available
- **Volume Management:** Survives container restarts

7.4 Load Balancing Strategy

Backend Load Balancing

- **Method:** Least connection (least_conn)
- **Health Checks:** Automatic backend instance health monitoring
- **Failover:** Automatic failover on backend failure
- **Rate Limiting:** 10 requests/second per IP (burst: 20)

Frontend Load Balancing

- **Method:** Round-robin
- **Health Checks:** HTTP health check endpoint
- **Static File Serving:** Nginx serves built React application

8. Provisioning

8.1 Provisioning Script (infra/provision.sh)

The provisioning script automates infrastructure setup with focus on:

Scalability

- Stateless service design
- Horizontal scaling support
- Resource limit configuration

- Load balancer ready

Security

- Network isolation (Docker networks)
- Credential management (.env with secure generation)
- Least-privilege access patterns
- Secure endpoint configuration

Extensibility

- Modular service architecture
- Easy to add new services
- Clear separation of concerns
- Infrastructure as Code ready

8.2 Provisioning Steps

1. Pre-flight Checks

- Verify Docker and Docker Compose installation
- Check Docker daemon status

2. Configuration Loading

- Load environment-specific configuration
- Set replica counts (min/max)

3. Credential Setup

- Generate JWT secret (32 bytes, base64)
- Generate database password (24 bytes, base64)
- Create .env file with secure permissions (600)

4. Compute Provisioning

- Build backend Docker image
- Tag for potential registry push

5. Database Provisioning

- Configure PostgreSQL container
- Set up persistent volumes
- Configure health checks

6. Storage Provisioning

- Configure object storage (future: S3, Azure Blob, GCS)

7. Authentication Configuration

- Configure JWT-based authentication
- Set up session management

8. Networking Setup

- Create isolated Docker network
- Configure service communication

9. Load Balancer Configuration

- Set up Nginx load balancers
- Configure health checks
- Set up rate limiting

10. Monitoring Setup

- Configure monitoring stack (future: Prometheus, Grafana)

8.3 Teardown Script (`infra/teardown.sh`)

Safely removes infrastructure:

- Stops all containers
- Option to preserve volumes (`--keep-volumes`)
- Option to remove volumes (default)
- Clean network cleanup

8.4 Deployment Scripts

- `infra/deploy.sh`: Deployment automation
 - `scripts/build.sh`: Build automation
 - `scripts/backup-db.sh`: Database backup
 - `scripts/restore-db.sh`: Database restore
 - `scripts/safe-clean.sh`: Safe cleanup operations
-

9. Database Schema

9.1 Tables

users

```
- id: SERIAL PRIMARY KEY
- email: VARCHAR(255) UNIQUE NOT NULL
- password_hash: VARCHAR(255) NOT NULL
- first_name: VARCHAR(100)
- last_name: VARCHAR(100)
- role: VARCHAR(50) DEFAULT 'user' CHECK (role IN ('user', 'admin'))
- is_active: BOOLEAN DEFAULT true
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

events

```
- id: SERIAL PRIMARY KEY
- title: VARCHAR(255) NOT NULL
- description: TEXT
- venue: VARCHAR(255)
- event_date: TIMESTAMP NOT NULL
- total_tickets: INTEGER NOT NULL CHECK (total_tickets >= 0)
- available_tickets: INTEGER NOT NULL CHECK (available_tickets >= 0)
- price: DECIMAL(10, 2) NOT NULL CHECK (price >= 0)
- image_url: VARCHAR(500)
- status: VARCHAR(50) DEFAULT 'upcoming' CHECK (status IN ('upcoming', 'live', 'completed', 'cancelled'))
- created_by: INTEGER REFERENCES users(id)
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

tickets

```

- id: SERIAL PRIMARY KEY
- event_id: INTEGER REFERENCES events(id) ON DELETE CASCADE
- user_id: INTEGER REFERENCES users(id) ON DELETE SET NULL
- title: VARCHAR(255) NOT NULL
- description: TEXT
- status: VARCHAR(50) DEFAULT 'open' CHECK (status IN ('open', 'in_progress', 'resolved', 'closed', 'purchased', 'cancelled'))
- priority: VARCHAR(50) DEFAULT 'medium' CHECK (priority IN ('low', 'medium', 'high', 'urgent'))
- ticket_type: VARCHAR(50) DEFAULT 'support' CHECK (ticket_type IN ('support', 'event'))
- quantity: INTEGER DEFAULT 1 CHECK (quantity > 0)
- purchase_price: DECIMAL(10, 2)
- purchase_date: TIMESTAMP
- created_by: VARCHAR(255)
- assigned_to: VARCHAR(255)
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

sessions

```

- id: SERIAL PRIMARY KEY
- user_id: INTEGER REFERENCES users(id) ON DELETE CASCADE
- token_hash: TEXT NOT NULL
- expires_at: TIMESTAMP NOT NULL
- is_revoked: BOOLEAN DEFAULT false
- revoked_at: TIMESTAMP
- device_info: TEXT
- ip_address: VARCHAR(45)
- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- last_used_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

9.2 Indexes

Performance indexes created on:

- `users.email` (unique index)
- `users.role`
- `events.status`
- `events.event_date`
- `tickets.event_id`
- `tickets.user_id`
- `tickets.status`
- `tickets.ticket_type`
- `sessions.user_id`
- `sessions.expires_at`
- `sessions.token_hash`
- `sessions.is_revoked`
- Composite index: `sessions(user_id, is_revoked)`

9.3 Constraints

- **Foreign Keys:** Cascade deletes for referential integrity
 - **Check Constraints:** Data validation at database level
 - **Unique Constraints:** Email uniqueness, token hash uniqueness
 - **NOT NULL Constraints:** Required fields enforcement
-

10. Security

10.1 Authentication Security

- **Password Hashing:** bcrypt with 10 salt rounds
- **JWT Tokens:** Signed tokens with expiration
- **Token Storage:** Secure token storage in frontend (localStorage/sessionStorage)
- **Session Management:** Database-backed session tracking
- **Token Revocation:** Support for token blacklisting

10.2 Authorization Security

- **Role-Based Access Control (RBAC):** User and Admin roles
- **Middleware Protection:** Route-level authentication/authorization
- **Self-Service Limits:** Users can only modify their own data (except admins)

10.3 API Security

- **Rate Limiting:**
 - General API: 100 requests per 15 minutes
 - Auth endpoints: 5 requests per 15 minutes
- **CORS Protection:** Whitelist of allowed origins
- **Security Headers:** Helmet middleware for XSS, CSRF protection
- **Input Validation:** express-validator for request validation
- **SQL Injection Prevention:** Parameterized queries (pg library)

10.4 Network Security

- **Network Isolation:** Docker network isolation
- **Port Exposure:** Only necessary ports exposed
- **Internal Communication:** Services communicate via private network
- **Load Balancer Security:** Rate limiting, security headers

10.5 Data Security

- **Encryption at Rest:** Database password protection
 - **Encryption in Transit:** HTTPS/TLS (configured at load balancer)
 - **Credential Management:** Environment variables, future: secrets management
 - **Data Validation:** Database-level constraints
-

11. Scalability

11.1 Horizontal Scaling

- **Stateless Design:** No session storage in application
- **JWT Authentication:** Token-based, no server-side session storage
- **Database Connection Pooling:** pg connection pool for efficient connections
- **Load Balancer:** Nginx load balancer distributes requests
- **Multiple Instances:** Support for multiple backend/frontend instances

11.2 Vertical Scaling

- **Resource Limits:** CPU and memory limits per container
- **Database Optimization:** Indexes for query performance
- **Connection Pooling:** Efficient database connection management

11.3 Performance Optimizations

- **Database Indexes:** Indexed queries for fast lookups
- **Pagination:** Efficient data loading with pagination

- **Query Optimization:** Optimized SQL queries
- **Connection Pooling:** Reuse database connections
- **Caching:** Future: Redis for caching

11.4 Scalability Features

- **Auto-scaling Ready:** Stateless design supports auto-scaling
 - **Load Balancing:** Automatic request distribution
 - **Health Checks:** Automatic unhealthy instance removal
 - **Failover:** Automatic failover on instance failure
-

12. Development & Operations

12.1 Development Workflow

1. Local Development

```
# Provision infrastructure
cd infra && ./provision.sh local

# Start services
docker-compose up -d

# View logs
docker-compose logs -f backend
```

2. Backend Development

```
cd backend
npm run dev # TypeScript with hot reload
```

3. Frontend Development

```
cd frontend
npm run dev # Vite dev server
```

12.2 Production Deployment

1. Build Process

```
# Build backend
cd backend && npm run build

# Build frontend
cd frontend && npm run build
```

2. Container Deployment

```
# Build and start
docker-compose up -d --build
```

3. Health Monitoring

```
# Check health
curl http://localhost:3000/health
```

12.3 Database Management

- **Migrations:** Automatic schema initialization on startup
 - **Seeding:** Default admin user and sample data
 - **Backups:** Scripts available for database backup/restore
-

13. API Endpoints Summary

Authentication Endpoints

- `POST /api/auth/register` - Register new user
- `POST /api/auth/login` - Login and get JWT token
- `POST /api/auth/logout` - Logout and invalidate token

User Endpoints

- `GET /api/users` - List users (admin only)
- `GET /api/users/:id` - Get user (self or admin)
- `POST /api/users` - Create user (admin only)
- `PUT /api/users/:id` - Update user (self or admin)
- `DELETE /api/users/:id` - Delete user (admin only)

Event Endpoints

- `GET /api/events` - List events (public, supports filtering)
- `GET /api/events/:id` - Get event by ID (public)
- `POST /api/events` - Create event (authenticated)
- `PUT /api/events/:id` - Update event (authenticated)
- `DELETE /api/events/:id` - Delete event (admin only)
- `POST /api/events/:eventId/purchase` - Purchase tickets (authenticated)

Ticket Endpoints

- `GET /api/tickets` - List tickets (with filters)
- `GET /api/tickets/:id` - Get ticket
- `POST /api/tickets` - Create support ticket
- `PUT /api/tickets/:id` - Update ticket
- `DELETE /api/tickets/:id` - Delete ticket

System Endpoints

- `GET /health` - Health check endpoint
 - `GET /` - API information endpoint
-

14. Production Considerations

14.1 Security Checklist

- ☐ Change all default passwords
- ☐ Use secrets management (AWS Secrets Manager, Azure Key Vault)
- ☐ Enable HTTPS/TLS at load balancer
- ☐ Configure CORS properly for production domains
- ☐ Implement comprehensive rate limiting
- ☐ Add request validation middleware

- ☐ Enable security headers
- ☐ Set up WAF (Web Application Firewall)

14.2 Infrastructure Checklist

- ☐ Use managed database (RDS, Cloud SQL, Azure Database)
- ☐ Use managed object storage (S3, Azure Blob, GCS)
- ☐ Configure external auth provider (Cognito, Auth0)
- ☐ Set up load balancer with SSL certificates
- ☐ Configure auto-scaling policies
- ☐ Set up monitoring and alerting (Prometheus, Grafana)
- ☐ Configure backup strategies (automated backups)
- ☐ Use secrets management service
- ☐ Configure network security (VPC, firewalls, security groups)

14.3 Operations Checklist

- ☐ Set up CI/CD pipelines
 - ☐ Configure log aggregation (ELK, CloudWatch, etc.)
 - ☐ Set up APM (Application Performance Monitoring)
 - ☐ Create runbooks for common operations
 - ☐ Document disaster recovery procedures
 - ☐ Set up automated testing (unit, integration, e2e)
 - ☐ Configure error tracking (Sentry, Rollbar)
-

15. Future Enhancements

15.1 Planned Features

- Payment gateway integration (Stripe, PayPal)
- Email notifications (event reminders, ticket confirmations)
- File upload for event images
- Advanced search and filtering
- User favorites/wishlist
- Event reviews and ratings
- Social media integration
- Mobile app (React Native)

15.2 Technical Improvements

- Redis caching layer
 - WebSocket support for real-time updates
 - GraphQL API option
 - Microservices architecture migration
 - Event sourcing for audit trail
 - Advanced analytics and reporting
 - Multi-tenant support
 - Internationalization (i18n)
-

Conclusion

The Ticketing System is a comprehensive, production-ready event management and ticket purchasing platform. It demonstrates modern software engineering practices including:

- **Clean Architecture:** Separation of concerns, maintainable code

- **Security:** JWT authentication, RBAC, rate limiting, input validation
- **Scalability:** Stateless design, horizontal scaling, load balancing
- **Reliability:** Health checks, automatic failover, transaction safety
- **Developer Experience:** TypeScript, hot reload, comprehensive tooling
- **Operations:** Docker-based deployment, infrastructure as code, monitoring ready

The system is designed to handle real-world production workloads while maintaining code quality, security, and scalability.

Document Version: 1.0.0

Last Updated: 2024

Maintained By: Development Team